

Contents

Day 30 — The AI Engineer’s future 1

What has changed? 1

The continuous thread: one project 2

Project: AI Research & Engineering Agent 2

The technical stack I’d use 3

Language 3

Models 3

AI infrastructure 4

Application infrastructure 4

Agent tooling 4

What I would deliberately NOT teach 4

The assessment system 5

The deeper curriculum I’d build for you 5

Track A — AI systems 6

Track B — Agent systems 6

Track C — AI economics 6

And there is one particularly important idea I’d make the centerpiece 6

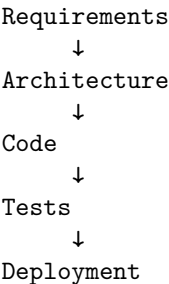
Day 30 — The AI Engineer’s future

The final day should be synthesis rather than another technology lesson.

Discuss:

What has changed?

2022:



2026:



Specification
↓
Context
↓
Agent(s)
↓
Generated implementation
↓
Automated verification
↓
Evaluation
↓
Human judgment
 ~ (loop back)

And potentially:

Human
↓
Intent
↓
Specification
↓
Agent swarm
↓
Software
↓
Automated evaluation
↓
Deployment

The engineer increasingly becomes the **designer of the system that produces software**, rather than the person who manually produces every line of software.

The continuous thread: one project

I would strongly recommend **one evolving project throughout the month**, rather than 30 disconnected tutorials.

For example:

Project: AI Research & Engineering Agent

Week 1:

Make it work

→ LLM + RAG + tools + evals

Week 2:

Make it robust

→ architecture + security + reliability + performance

Week 3:

Make the agent build it

→ coding agents + specs + verification + multi-agent workflows

Week 4:

Make it valuable

→ product definition + users + MVP + deployment

By Day 30 you have something substantially more valuable than a collection of notebooks.

The technical stack I'd use

I would deliberately keep the stack relatively narrow.

Language

Python + TypeScript

Python for:

- AI/ML
- evaluation
- data
- experimentation

TypeScript for:

- application layer
- APIs
- frontend

Models

Use multiple model providers rather than becoming dependent on one.

Learn the **model abstraction**, not just one API.

AI infrastructure

Learn:

- model APIs
- embeddings
- vector search
- structured outputs
- tool calling
- RAG
- agent frameworks

But don't hide the underlying mechanics behind frameworks.

Application infrastructure

- Git
- Docker
- PostgreSQL
- Redis
- object storage
- REST APIs
- queues
- cloud deployment

Agent tooling

Use **at least two different coding agents** and compare their workflows.

The objective isn't learning product-specific keystrokes. It's learning:

context → specification → planning → execution → verification → iteration.

What I would deliberately NOT teach

This is important.

A one-month bootcamp can easily become an enormous catalog of AI technologies.

I would **not** spend significant time on:

- training an LLM from scratch
- transformer implementation from scratch
- mathematical derivation of attention
- obscure agent frameworks
- endless prompt-engineering tricks

- fine-tuning unless the project requires it
- memorizing APIs
- chasing every new model
- building toy chatbots
- “100 LLM projects”

Those are interesting, but they don’t map strongly onto Ng’s four skills.

The assessment system

I’d make the bootcamp unusually rigorous.

Every project gets scored on five dimensions:

Dimension	Weight
AI system quality	20%
Software engineering	20%
Agent utilization	20%
Evaluation/reliability	20%
Product judgment	20%

And I’d require **evidence**, not claims.

For example:

“Our RAG system is good.”

is meaningless.

Instead:

“On a 500-question evaluation set, retrieval recall@5 increased from 71% to 89%, grounded-answer accuracy increased from 76% to 91%, while p95 latency increased by 140 ms.”

That is AI engineering.

The deeper curriculum I’d build for you

Given your existing systems/ML background, I would actually make **your version considerably more advanced** than this baseline.

I would add three tracks running underneath the four weeks:

Track A — AI systems

Go deeper into:

- inference
- KV cache
- batching
- quantization
- model routing
- GPU/ANE/NPU utilization
- distributed inference
- latency/throughput
- memory bandwidth

Track B — Agent systems

Go deeper into:

- agent harness architecture
- context management
- planning
- tool protocols
- memory
- subagents
- verifiers
- agent evaluation
- autonomous loops
- multi-agent coordination

Track C — AI economics

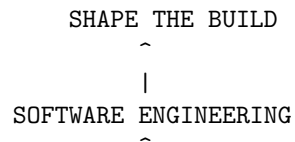
Study:

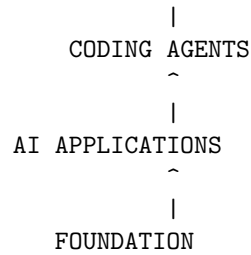
$$\text{AI value} = \frac{\text{capability} \times \text{adoption}}{\text{latency} \times \text{cost} \times \text{failure rate}}$$

This connects technical architecture to the business/product decision.

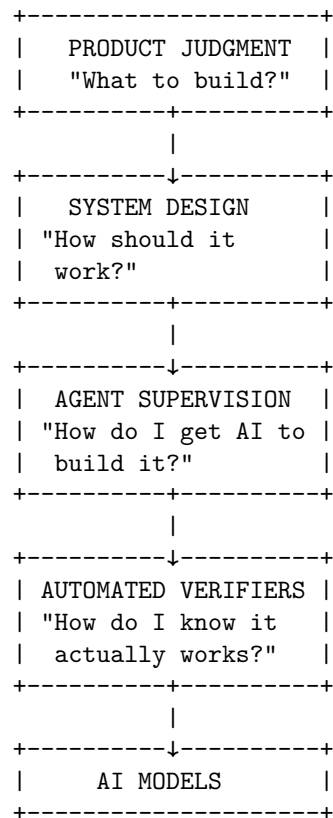
And there is one particularly important idea I'd make the centerpiece

The post's four categories can actually be arranged as a **hierarchy**:





But I think the real future hierarchy is closer to:



That is the curriculum I'd optimize for.

And given the kinds of questions you've been asking me recently—agentic harnesses, test-time training, scaling laws, AI accelerators, ML systems, and the changing role of software engineering—I would push this beyond a generic AI-engineering bootcamp into something closer to **“AI Systems Engineering 2026: From LLM Applications to Agentic Software Engineering.”** That would make the month much more intellectually demanding and much more aligned with where your existing expertise gives you an advantage.